Lucas Valandro da Rocha

Resolvendo o ARC-AGI-1 com LLMs — Relatório do Nível 1

IA002 · Explorando o Potencial de Agentes de IA Generativa · Especialização em IA Avançada, UFRGS

1. Objetivo

Comparar duas formas de usar um LLM para resolver tarefas do ARC-AGI-1, cada uma com dois níveis de raciocínio. Na abordagem **fim a fim**, o modelo recebe os pares de treino e escreve diretamente a grade de resposta. Na abordagem **geração de código**, o modelo escreve uma função Python `transform(grid)`, que é executada nos exemplos de treino e na entrada de teste. A comparação considera o número de tarefas resolvidas, o custo em tokens e o tempo.

2. Configuração

- **Modelo:** Gemini 3.5 Flash Lite (`gemini-3.5-flash-lite`), pela API do Google, executado no notebook `arc_solver_nivel1.ipynb` no Google Colab, com pausa de 1 s entre chamadas.
- **Quatro configurações:** fim a fim com raciocínio mínimo, fim a fim com raciocínio alto, código com raciocínio mínimo e código com raciocínio alto. Cada configuração faz uma única chamada por tarefa, totalizando 80 chamadas.
- **20 tarefas:** 10 do conjunto de treino (3aa6fb7a, 44d8ac46, 44f52bb0, 007bbfb7, 25ff71a9, 0a938d79, 1cf80156, 2013d3e2, 32597951, 9565186b) e 10 do conjunto de avaliação (9a4bb226, aab50785, d47aa2ff, f5aa3634, 93b4f4b3, e9b4f6fc, 7c9b52a0, c658a4bd, 50aad11f, 7d18a6fb).
- **Critério:** uma tarefa é resolvida quando todas as grades de teste produzidas são exatamente iguais às respostas esperadas.

3. Resultados

A Tabela 1 resume o arquivo `resultados.csv`. Os tokens e o tempo são somas das 20 tarefas.

Abordagem	Raciocínio	Resolvidas	Treino	Avaliação	Entrada	Raciocínio	Saída	Tempo (s)
Fim a fim	mínimo	3/20	3/10	0/10	53 706	0	6 346	96,5
Fim a fim	alto	18/20	10/10	8/10	53 706	180 967	5 640	425,3
Código	mínimo	9/20	6/10	3/10	53 426	0	55 304	172,4
Código	alto	18/20	9/10	9/10	53 426	222 127	7 951	580,9

Tabela 1: Tarefas resolvidas (total e por conjunto), tokens de entrada, raciocínio e saída, e tempo total.

- O raciocínio alto foi o fator decisivo. Ele elevou o fim a fim de 3 para 18 tarefas e o código de 9 para 18.
- Com raciocínio mínimo, o código superou o fim a fim (9 contra 3). Porém, gerou 55 304 tokens de saída, e três programas falharam com erros de Python (`SyntaxError`, `IndentationError` e `NameError`).
- Com raciocínio alto, o código acertou todos os exemplos de treino em 18 tarefas. Em 17 delas o teste também foi resolvido.
- O raciocínio alto aumentou o tempo total em cerca de 4 vezes no fim a fim e 3 vezes no código.

4. Exemplos

Tarefa resolvida: 3aa6fb7a (treino). Cada grade contém figuras em forma de "L" com três células azul-claras (cor 8) dentro de um bloco 2×2. A regra é pintar de azul (cor 1) a célula vazia que completa o bloco. Com raciocínio mínimo, as duas abordagens erraram. Com raciocínio alto, as duas resolveram a tarefa. O raciocínio extra permitiu identificar a forma e a posição da célula que falta, e não apenas copiar padrões de cor.

Tarefa não resolvida: 44f52bb0 (treino), código com raciocínio alto. A entrada é uma grade 3×3 e a saída é uma única célula: 1 se a figura é simétrica em relação ao eixo vertical, e 7 caso contrário. O programa gerado acertou os 7 exemplos de treino (`treino_ok = True`), mas errou o teste. Isso indica que o modelo encontrou uma regra que se ajusta aos exemplos, mas não é a regra verdadeira. Esse é um sobreajuste típico quando há poucos exemplos. A mesma tarefa foi resolvida pelo fim a fim com raciocínio alto, o que mostra que acertar o treino não garante acertar o teste.

5. Conclusão

As duas configurações com raciocínio alto empataram com 18 de 20 tarefas resolvidas. A configuração com melhor desempenho é o **fim a fim com raciocínio alto**, pois obtém o mesmo resultado com menor custo: 240 313 tokens no total (53 706 de entrada, 180 967 de raciocínio e 5 640 de saída), cerca de 13 350 tokens por tarefa resolvida, e 425,3 s. O código com raciocínio alto usou 283 504 tokens (cerca de 15 750 por tarefa resolvida) e 580,9 s. Em compensação, ele resolveu uma tarefa a mais do conjunto de avaliação (9 contra 8) e permite verificar o programa nos exemplos de treino antes de responder. Os resultados vêm de uma única execução por tarefa. Por isso, uma diferença de uma ou duas tarefas pode ser apenas variação do modelo.

6. Uso de IA

Um assistente de IA (Claude Code, usado no lugar do ChatGPT) foi utilizado para calcular os totais a partir do arquivo `resultados.csv` e redigir este relatório, e os números foram conferidos com a tabela do notebook.